

LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF320803)
MINGGU 5 ACARA 10
SEMESTER 3



Tugas :
Inheritance, Base Controller, Base
Model, Repository Pattern

Disusun Oleh:
Uaryl Engno Laurenca (E41250317)

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

A. PENDAHULUAN

Pada aplikasi berbasis MVC, banyak Controller memiliki fungsi yang sama, misalnya untuk menampilkan halaman (`view()`) dan mengarahkan pengguna ke halaman lain (`redirect()`). Apabila setiap Controller menuliskan kedua method tersebut secara berulang, kode menjadi tidak efisien dan sulit dipelihara. Konsep Inheritance (pewarisan) mengatasi hal ini dengan membuat sebuah `BaseController` yang menyediakan method-method umum, kemudian diwarisi (`extends`) oleh Controller lain yang membutuhkannya.

Menariknya, pada praktikum Acara 10 ini ditemukan bahwa kedua konsep utama yang diminta Inheritance melalui `BaseController` dan Repository Pattern melalui `MahasiswaRepository` sudah diterapkan sejak Acara 8 dan Acara 9, sebagai bagian dari pembangunan struktur aplikasi CRUD Mahasiswa yang bertahap. Acara 10 ini kemudian berfungsi untuk memverifikasi bahwa kedua konsep tersebut telah diterapkan secara benar dan sesuai dengan alur kerja yang dijelaskan pada dasar teori.

B. TUJUAN PRAKTIKUM

1. Memahami konsep Inheritance dan manfaatnya dalam mengurangi duplikasi kode (prinsip *DRY – Don't Repeat Yourself*).
2. Memverifikasi bahwa `BaseController` berisi method `view()` dan `redirect()` sebagai method umum yang dapat diwarisi oleh Controller lain.
3. Memverifikasi bahwa `MahasiswaController` mewarisi (`extends`) `BaseController` dan menggunakan method warisan tersebut untuk menampilkan halaman dan melakukan `redirect`.
4. Memverifikasi bahwa `MahasiswaRepository` menangani seluruh operasi database (tampil, tambah, ubah, hapus data mahasiswa) sesuai prinsip Repository Pattern.
5. Memastikan `MahasiswaController` sepenuhnya bebas dari query SQL, karena seluruh akses data dilakukan melalui `MahasiswaRepository`.
6. Menguji aplikasi untuk memastikan fitur tampil, tambah, ubah, dan hapus data tetap berjalan dengan baik.

C. DASAR TEORI

Inheritance (Pewarisan) adalah konsep OOP yang memungkinkan sebuah class mewarisi atribut dan method dari class lain. Class yang mewarisi disebut *child class*, seperti `MahasiswaController`, sedangkan class yang diwarisi disebut *parent class* atau *base class*, seperti `BaseController`. Dengan inheritance, method umum seperti `view()` dan `redirect()` cukup ditulis satu kali di `BaseController` dan dapat digunakan oleh seluruh Controller yang menggunakan `extends` melalui `$this->view(...)` atau `$this->redirect(...)`. Penerapan inheritance dapat mengurangi duplikasi kode (*DRY*), mempermudah pemeliharaan aplikasi, serta membuat perubahan pada method umum cukup dilakukan di `BaseController` tanpa perlu mengubah seluruh Controller turunannya.

Repository Pattern adalah pola desain yang digunakan untuk memisahkan logika akses data, khususnya query SQL, dari logika aplikasi pada Controller. Seluruh proses pengambilan, penambahan, perubahan, dan penghapusan data ditempatkan di dalam class Repository. Dengan demikian, Controller cukup memanggil method yang tersedia, seperti `$repository->all()` atau `$repository->create($data)`, tanpa perlu mengetahui bagaimana query SQL dijalankan. Penerapan Repository Pattern membuat Controller lebih sederhana dan fokus mengatur alur aplikasi, memisahkan logika bisnis dari akses database, mempermudah pengujian (*testing*), serta memudahkan perubahan struktur database atau DBMS karena penyesuaian dapat dilakukan pada Repository tanpa harus mengubah Controller.

Alur kerja Inheritance dimulai dari `BaseController` sebagai class dasar yang menyediakan method umum seperti `view()` dan `redirect()`. Selanjutnya, `MahasiswaController` menggunakan `extends` untuk mewarisi method tersebut sehingga dapat langsung menggunakannya tanpa menulis ulang kode yang sama. Sementara itu, alur kerja Repository Pattern dimulai ketika Browser mengirimkan request kepada Controller. Controller kemudian memanggil method yang tersedia pada Repository, bukan menjalankan query SQL secara langsung. Repository menjalankan query ke Database melalui PDO, kemudian mengembalikan hasil data kepada Controller. Selanjutnya, Controller mengirimkan data tersebut ke View, dan View mengolahnya menjadi halaman HTML yang kemudian ditampilkan kembali kepada pengguna melalui Browser.

D. HASIL DAN PEMBAHASAN

1. Verifikasi BaseController

```
ACARA10 > app > Core > Controller.php
1  <?php
2
3  namespace App\Core;
4
5  class Controller
6  {
7      protected function view(string $view, array $data = [], string $layout = 'main'): void
8      {
9          extract($data);
10         $content = __DIR__ . '/../Views/' . $view . '.php';
11         require __DIR__ . '/../Views/layouts/' . $layout . '.php';
12     }
13
14     protected function redirect(string $path): void
15     {
16         header('Location: ' . BASE_URL . $path);
17         exit;
18     }
19
20     protected function flash(string $type, string $message): void
21     {
22         $_SESSION['flash'] = ['type' => $type, 'message' => $message];
23     }
24 }
```

`app/Core/Controller.php`, yang telah dibuat sejak Acara 8, ternyata sudah persis sesuai dengan konsep BaseController yang diminta modul ini bahkan penamaan class-nya

sama, yaitu Controller. Class ini berisi method view() untuk me-render halaman dengan data yang dikirim, dan redirect() untuk mengarahkan pengguna ke URL lain, keduanya dideklarasikan sebagai protected agar hanya bisa dipanggil dari dalam class itu sendiri atau class turunannya.

2. Verifikasi MahasiswaController Mewarisi BaseController

```
1 <?php
2
3 namespace App\Controllers;
4
5 use App\Core\Controller;
6 use App\Repositories\MahasiswaRepository;
7 use App\Models\Mahasiswa;
8 use App\Models\ProdiModel;
9
10 class MahasiswaController extends Controller
11 {
12     private MahasiswaRepository $repo;
13     private ProdiModel $prodiModel;
14
15     // Repository disuntikkan dari luar (lihat public/index.php), controller tidak lagi
16     // membuat koneksi database atau Repository-nya sendiri
17     public function __construct(MahasiswaRepository $repo)
18     {
19         $this->repo = $repo;
20         $this->prodiModel = new ProdiModel(); // hanya untuk dropdown, tetap dari Acara 8
21     }
22
23     public function index(): void
24     {
25         $keyword = trim($_GET['q'] ?? '');
26         $mahasiswa = $this->repo->all($keyword);
27
28         $this->view('mahasiswa/index', [
29             'title' => 'Data Mahasiswa',
30             'mahasiswa' => $mahasiswa,
31             'keyword' => $keyword,
32         ]);
33     }
34
35     public function create(): void
36     {
37         $this->view('mahasiswa/create', [
38             'title' => 'Tambah Mahasiswa',
39             'prodiList' => $this->prodiModel->all(),
40         ]);
41     }
42
43     public function store(): void
44     {
45         try {
46             // Validasi (NIM angka, nama tidak kosong) terjadi otomatis di constructor Mahasiswa
47             $mhs = new Mahasiswa([
48                 $_POST['nim'] ?? '',
49                 $_POST['nama'] ?? '',
50                 $_POST['email'] ?? '',
51                 (int) ($_POST['prodi_id'] ?? 0),
52                 (int) ($_POST['angkatan'] ?? date('Y'))
53             ]);
54             $this->repo->create($mhs);
55             $this->flash('success', 'Data mahasiswa berhasil ditambahkan.');
```

MahasiswaController sudah dideklarasikan sebagai class MahasiswaController extends Controller sejak Acara 8, dan seluruh method-nya (index(), create(), store(), edit(), update(), destroy()) memanggil \$this->view(...) dan \$this->redirect(...) yang diwarisi dari BaseController, tanpa ada satu pun require_once atau header() yang ditulis manual di dalam Controller ini.

3. Verifikasi MahasiswaRepository Menangani Seluruh Operasi Database

```
1 <?php
2
3 namespace App\Repositories;
4
5 use App\Models\Mahasiswa;
6 use PDO;
7
8 class MahasiswaRepository
9 {
10     private PDO $pdo;
11
12     // Constructor Dependency Injection: menerima koneksi dari luar (Database::getInstance()),
13     // bukan membuat koneksinya sendiri -> lebih fleksibel dan mudah diuji (testable)
14     public function __construct(PDO $pdo)
15     {
16         $this->pdo = $pdo;
17     }
18
19     public function all(string $keyword = ''): array
20     {
21         $sql = "SELECT m.*, p.nama AS prodi_nama
22             FROM mahasiswa m
23             JOIN prodi p ON m.prodi_id = p.id";
24         $params = [];
25
26         if ($keyword != '') {
27             $sql .= " WHERE m.nama LIKE :keyword OR m.nim LIKE :keyword";
28             $params['keyword'] = '%' . $keyword . '%';
29         }
30         $sql .= " ORDER BY m.nama ASC";
31
32         $stmt = $this->pdo->prepare($sql);
33         $stmt->execute($params);
34
35         // Ubah setiap baris array hasil query menjadi objek Mahasiswa
36         return array_map(fn($row) => $this->mapToEntity($row), $stmt->fetchAll());
37     }
38
39     public function find(int $id): ?Mahasiswa
40     {
41         $stmt = $this->pdo->prepare("SELECT * FROM mahasiswa WHERE id = :id");
42         $stmt->execute(['id' => $id]);
43         $row = $stmt->fetch();
44         return $row ? $this->mapToEntity($row) : null;
45     }
46
47     public function create(Mahasiswa $mhs): void
48     {
49         $stmt = $this->pdo->prepare(
50             "INSERT INTO mahasiswa (nim, nama, email, prodi_id, angkatan)
51             VALUES (:nim, :nama, :email, :prodi_id, :angkatan)"
52         );
53         $stmt->execute([
54             'nim' => $mhs->getNim(),
55             'nama' => $mhs->getNama(),
56             'email' => $mhs->getEmail(),
57             'prodi_id' => $mhs->getProdiId(),
58             'angkatan' => $mhs->getAngkatan(),
59         ]);
60     }
61
62     public function update(Mahasiswa $mhs): void
63     {
64         $stmt = $this->pdo->prepare(
65             "UPDATE mahasiswa SET nim = :nim, nama = :nama, email = :email, prodi_id = :prodi_id, angkatan = :angkatan
66             WHERE id = :id"
67         );
68         $stmt->execute([
69             'nim' => $mhs->getNim(),
70             'nama' => $mhs->getNama(),
71             'email' => $mhs->getEmail(),
72             'prodi_id' => $mhs->getProdiId(),
73             'angkatan' => $mhs->getAngkatan(),
74             'id' => $mhs->getId(),
75         ]);
76     }
77
78     public function delete(int $id): void
79     {
80         $stmt = $this->pdo->prepare("DELETE FROM mahasiswa WHERE id = :id");
81         $stmt->execute(['id' => $id]);
82     }
83
84     // Helper: ubah array hasil query menjadi objek Mahasiswa
85     private function mapToEntity(array $row): Mahasiswa
86     {
87         return new Mahasiswa(
88             $row['nim'],
89             $row['nama'],
90             $row['email'] ?? '',
91             (int) $row['prodi_id'],
92             (int) $row['angkatan'],
93             (int) $row['id'],
94             $row['prodi_nama'] ?? null
95         );
96     }
97 }
```

MahasiswaRepository, yang dibangun pada Acara 9, memiliki method lengkap untuk seluruh operasi CRUD: all(), find(), create(), update(), dan delete(). Seluruh perintah SQL termasuk JOIN dengan tabel prodi berada di dalam Repository ini, sehingga sepenuhnya memenuhi prinsip Repository Pattern.

4. Verifikasi Controller Bebas dari Query SQL

Pemeriksaan menyeluruh terhadap MahasiswaController memastikan tidak ada satu pun query SQL yang dituliskan langsung di dalamnya. Setiap method Controller hanya memanggil method pada \$this->repo, misalnya:

php

```
$mahasiswa = $this->repo->all($keyword);
```

```
$this->view('mahasiswa/index', ['mahasiswa' => $mahasiswa]);
```

Pendekatan ini konsisten dengan alur kerja Repository Pattern pada dasar teori: Controller → Repository → Database, tanpa ada logika akses data yang bocor ke Controller.

5. Pengujian Fungsi Aplikasi

Karena BaseController dan MahasiswaRepository sudah diterapkan sejak awal, pengujian pada Acara 10 ini berfokus pada memastikan seluruh fitur CRUD mahasiswa tetap berjalan sebagaimana mestinya:

- Tampil data halaman /mahasiswa menampilkan daftar mahasiswa lengkap dengan nama prodi (hasil JOIN), dirender lewat \$this->view() yang diwarisi dari BaseController.
- Tambah data form pada /mahasiswa/create berhasil menyimpan data baru melalui store() → \$this->repo->create(), lalu di-redirect lewat \$this->redirect().
- Ubah data — form pada /mahasiswa/{id}/edit berhasil memperbarui data melalui update() → \$this->repo->update().
- Hapus data tombol hapus pada daftar mahasiswa berhasil menghapus data melalui destroy() → \$this->repo->delete().

SI Akademik

MahasiswaProdiMata Kuliah

Data Mahasiswa

+ Tambah Mahasiswa

Cari nama atau NIM...

Cari

NIM	Nama	Email	Prodi	Angkatan	Aksi
2401001	Budi Santoso	budi@email.com	Teknik Informatika	2024	EditHapus
2402001	Citra Lestari	citra@email.com	Sistem Informasi	2024	EditHapus
230011	uaryl	dpa71845@gmail.com	Teknik Informatika	2026	EditHapus

SI Akademik
Mahasiswa Prodi Mata Kuliah

Tambah Mahasiswa

NIM

230011

Nama

uaryl

Email

dpa71845@gmail.com

Prodi

Teknik Informatika

Angkatan

2026

Batal Simpan

SI Akademik
Mahasiswa Prodi Mata Kuliah

Data Mahasiswa

+ Tambah Mahasiswa

Cari nama atau NIM... Cari

NIM	Nama	Email	Prodi	Angkatan	Aksi
2401001	Budi Santoso	budi@email.com	Teknik Informatika	2024	Edit Hapus
2402001	Citra Lestari	citra@email.com	Sistem Informasi	2024	Edit Hapus
230011	Uaryl Engno Laurencia	dpa71845@gmail.com	Teknik Informatika	2026	Edit Hapus

Seluruh pengujian berjalan sukses tanpa error, membuktikan bahwa BaseController dan MahasiswaRepository yang telah dibangun bertahap sejak Acara 8–9 benar-benar berfungsi dan konsisten dengan alur MVC yang diharapkan modul.

E. KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Inheritance melalui BaseController (app/Core/Controller.php) efektif dalam mengurangi duplikasi kode karena method view() dan redirect() cukup ditulis satu kali dan dapat digunakan oleh MahasiswaController maupun Controller lainnya melalui extends, sesuai dengan prinsip *DRY (Don't Repeat Yourself)*. Selain itu, penerapan Repository Pattern melalui MahasiswaRepository berhasil memisahkan Controller dari logika akses database, sehingga seluruh query SQL berada di dalam Repository dan Controller menjadi lebih ringkas, mudah dibaca, serta berfokus pada alur logika aplikasi. Kombinasi Inheritance dan Repository Pattern yang telah diterapkan sejak Acara 8–9 juga menghasilkan struktur kode yang konsisten dengan alur MVC, yaitu Controller → Repository → Database → View, sehingga aplikasi lebih mudah dikembangkan ketika membutuhkan Controller atau Repository baru untuk resource lain, seperti Prodi atau Mata Kuliah. Pengujian terhadap fitur tampil, tambah, ubah, dan hapus data mahasiswa menunjukkan bahwa struktur BaseController dan MahasiswaRepository dapat berjalan dengan baik tanpa memerlukan perubahan tambahan untuk memenuhi tujuan Acara 10. Dengan demikian, praktikum ini menunjukkan bahwa perencanaan dan penerapan arsitektur yang konsisten sejak awal dapat menghasilkan kode yang lebih rapi, fleksibel, mudah dipelihara, serta memungkinkan pengembangan fitur berikutnya dilakukan tanpa memerlukan refactoring besar.

